

**Московский государственный технический
университет им. Н. Э. Баумана**

Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»
Отчет по рубежному контролю №2
«Вариант А, 4»

Выполнил:
Студент группы ИУ5-31Б
Горячева Софья

Проверил:
Гапанюк Ю. Е.

2025 г.

Условия рубежного контроля №2 по курсу ПиК ЯП

Рубежный контроль представляет собой разработку тестов на языке Python.

- 1) Проведите рефакторинг текста программы рубежного контроля №1 таким образом, чтобы он был пригоден для модульного тестирования.
- 2) Для текста программы рубежного контроля №1 создайте модульные тесты с применением TDD - фреймворка (3 теста).

Листинг кода

```
1  from operator import itemgetter
2  import unittest
3
4  class Computer:
5      def __init__(self, id, name, price, display_class_id):
6          self.id = id
7          self.name = name
8          self.price = price
9          self.display_class_id = display_class_id
10
11  class DisplayClass:
12      def __init__(self, id, name):
13          self.id = id
14          self.name = name
15
16  class ComputerDisplayClass:
17      def __init__(self, display_class_id, computer_id):
18          self.display_class_id = display_class_id
19          self.computer_id = computer_id
20
21  def create_one_to_many(display_classes, computers):
22      """Логика для связи один-ко-многим"""
23      return [(c.name, c.price, dc.name)
24              for dc in display_classes
25              for c in computers
26              if c.display_class_id == dc.id]
27
28  def create_many_to_many_temp(display_classes, computers_display_classes):
29      """Логика для временной связи многие-ко-многим"""
30      return [(dc.name, cdc.display_class_id, cdc.computer_id)
31              for dc in display_classes
32              for cdc in computers_display_classes
33              if dc.id == cdc.display_class_id]
34
35  def create_many_to_many(many_to_many_temp, computers):
36      """Логика для связи многие-ко-многим"""
```

```

37         return [(c.name, c.price, dc_name)
38                 for dc_name, dc_id, computer_id in many_to_many_temp
39                 for c in computers if c.id == computer_id]
40
41 def sort_computers_by_class(one_to_many):
42     """Логика задания А1"""
43     return sorted(one_to_many, key=itemgetter(2))
44
45 def calculate_total_prices(one_to_many, display_classes):
46     """Логика задания А2"""
47     display_classes_total_price = []
48     for dc in display_classes:
49         dc_computers = list(filter(lambda i: i[2] == dc.name, one_to_many))
50         if len(dc_computers) > 0:
51             total_price = sum([price for _, price, _ in dc_computers])
52             display_classes_total_price.append((dc.name, total_price))
53     return sorted(display_classes_total_price, key=itemgetter(1), reverse=True)
54
55 def filter_departments_with_computers(many_to_many, display_classes):
56     """Логика задания А3"""
57     departments_with_computers = {}
58     for dc in display_classes:
59         if 'отдел' in dc.name:
60             dc_computers = list(filter(lambda i: i[2] == dc.name, many_to_many))
61             computer_names = [name for name, _, _ in dc_computers]
62             departments_with_computers[dc.name] = computer_names
63     return departments_with_computers
64
65 def main():
66     # Исходные данные
67     display_classes = [
68         DisplayClass(1, 'отдел игровых компьютеров'),
69         DisplayClass(2, 'архивный отдел офисной техники'),
70         DisplayClass(3, 'бухгалтерия'),
71         DisplayClass(11, 'отдел графических станций'),
72
73         DisplayClass(22, 'архивный отдел серверов'),
74         DisplayClass(33, 'отдел тестирования'),
75     ]
76
77     computers = [
78         Computer(1, 'ASUS ROG', 25000, 1),
79         Computer(2, 'HP Office', 35000, 2),
80         Computer(3, 'Apple MacBook', 45000, 3),
81         Computer(4, 'Dell Precision', 35000, 3),
82         Computer(5, 'Lenovo ThinkPad', 25000, 3),
83     ]
84
85     computers_display_classes = [
86         ComputerDisplayClass(1, 1),
87         ComputerDisplayClass(2, 2),
88         ComputerDisplayClass(3, 3),
89         ComputerDisplayClass(3, 4),
90         ComputerDisplayClass(3, 5),
91         ComputerDisplayClass(11, 1),
92         ComputerDisplayClass(22, 2),
93         ComputerDisplayClass(33, 3),
94         ComputerDisplayClass(33, 4),
95         ComputerDisplayClass(33, 5),
96     ]
97
98     # Создание связей с помощью вынесенных функций
99     one_to_many = create_one_to_many(display_classes, computers)
100     many_to_many_temp = create_many_to_many_temp(display_classes, computers_display_classes)
101     many_to_many = create_many_to_many(many_to_many_temp, computers)
102
103     print('Задание А1')
104     print('Список всех связанных компьютеров и дисплейных классов, отсортированный по дисплейным классам:')
105     result_a1 = sort_computers_by_class(one_to_many)
106     for item in result_a1:
107         print(f" {item[2]}: {item[0]} - {item[1]} руб.")

```

```

107
108     print('\nЗадание A2')
109     print('Список дисплейных классов с суммарной стоимостью компьютеров в каждом классе, отсортированный по суммарной стоимости:')
110     result_a2 = calculate_total_prices(one_to_many, display_classes)
111     for item in result_a2:
112         print(f" {item[0]}: {item[1]} руб.")
113
114     print('\nЗадание A3')
115     print('Список всех дисплейных классов, у которых в названии присутствует слово "отдел", и список компьютеров в них:')
116     result_a3 = filter_departments_with_computers(many_to_many, display_classes)
117     for department, computers_list in result_a3.items():
118         print(f" {department}: {' '.join(computers_list)}")
119
120
121 class TestRK1Functions(unittest.TestCase):
122     """Тесты для функций РК1"""
123
124     def setUp(self):
125         """Тестовые данные"""
126         self.display_classes = [
127             DisplayClass(1, 'отдел игровых компьютеров'),
128             DisplayClass(2, 'архивный отдел офисной техники'),
129             DisplayClass(3, 'бухгалтерия'),
130         ]
131
132         self.computers = [
133             Computer(1, 'ASUS ROG', 25000, 1),
134             Computer(2, 'HP Office', 35000, 2),
135             Computer(3, 'Apple MacBook', 45000, 3),
136         ]
137
138         self.computers_display_classes = [
139             ComputerDisplayClass(1, 1),
140             ComputerDisplayClass(2, 2),
141             ComputerDisplayClass(3, 3),
142             ComputerDisplayClass(1, 2),
143         ]
144
145     def test_sort_computers_by_class(self):
146         """Тест 1: Проверка сортировки компьютеров по классам (Задание A1)"""
147         one_to_many = create_one_to_many(self.display_classes, self.computers)
148         result = sort_computers_by_class(one_to_many)
149
150         # Проверяем сортировку по имени класса
151         expected_order = ['архивный отдел офисной техники', 'бухгалтерия', 'отдел игровых компьютеров']
152         actual_order = [item[2] for item in result]
153
154         self.assertEqual(actual_order, expected_order)
155         self.assertEqual(len(result), 3)
156         self.assertEqual(result[0][0], 'HP Office') # Первый компьютер в отсортированном списке
157
158     def test_calculate_total_prices(self):
159         """Тест 2: Проверка расчета суммарной стоимости (Задание A2)"""
160         one_to_many = create_one_to_many(self.display_classes, self.computers)
161         result = calculate_total_prices(one_to_many, self.display_classes)
162
163         # Проверяем правильность расчетов
164         expected_results = {
165             'бухгалтерия': 45000,
166             'архивный отдел офисной техники': 35000,
167             'отдел игровых компьютеров': 25000
168         }
169
170         self.assertEqual(len(result), 3)
171
172         for class_name, total_price in result:
173             self.assertEqual(total_price, expected_results[class_name])
174
175         # Проверяем сортировку по убыванию цены

```

```

176         self.assertTrue(result[0][1] >= result[1][1] >= result[2][1])
177
178     def test_filter_departments_with_computers(self):
179         """Тест 3: Проверка фильтрации отделов (Задание A3)"""
180         many_to_many_temp = create_many_to_many_temp(self.display_classes, self.computers_display_classes)
181         many_to_many = create_many_to_many(many_to_many_temp, self.computers)
182         result = filter_departments_with_computers(many_to_many, self.display_classes)
183
184         # Проверяем, что выбраны только отделы
185         self.assertIn('отдел игровых компьютеров', result)
186         self.assertIn('архивный отдел офисной техники', result)
187         self.assertNotIn('бухгалтерия', result)
188
189         # Проверяем состав компьютеров в отделах
190         self.assertEqual(len(result['отдел игровых компьютеров']), 2)
191         self.assertIn('ASUS ROG', result['отдел игровых компьютеров'])
192         self.assertIn('HP Office', result['отдел игровых компьютеров'])
193
194         self.assertEqual(len(result['архивный отдел офисной техники']), 1)
195         self.assertIn('HP Office', result['архивный отдел офисной техники'])
196
197
198 if __name__ == '__main__':
199     print("=== ВАШ PK1 С ТЕСТАМИ ===")
200     print("\n--- Запуск основной программы ---")
201     main()
202
203     print("\n--- Запуск модульных тестов ---")
204     unittest.main(argv=[''], exit=False, verbosity=2)

```

Результат выполнения

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
sofagoraceva@MacBook-Air-Sofa Goryacheva_2025_Labs % /usr/local/bin/python3 /Users/sofagoraceva/Goryacheva_2025_Labs/Goryacheva_2025_Labs/RK2/RK2.py	<pre> === ВАШ PK1 С ТЕСТАМИ === --- Запуск основной программы --- Задание A1 Список всех связанных компьютеров и дисплейных классов, отсортированный по дисплейным классам: архивный отдел офисной техники: HP Office - 35000 руб. бухгалтерия: Apple MacBook - 45000 руб. бухгалтерия: Dell Precision - 35000 руб. бухгалтерия: Lenovo ThinkPad - 25000 руб. отдел игровых компьютеров: ASUS ROG - 25000 руб. Задание A2 Список дисплейных классов с суммарной стоимостью компьютеров в каждом классе, отсортированный по суммарной стоимости: бухгалтерия: 105000 руб. архивный отдел офисной техники: 35000 руб. отдел игровых компьютеров: 25000 руб. Задание A3 Список всех дисплейных классов, у которых в названии присутствует слово "отдел", и список компьютеров в них: отдел игровых компьютеров: ASUS ROG архивный отдел офисной техники: HP Office отдел графических станций: ASUS ROG архивный отдел серверов: HP Office отдел тестирования: Apple MacBook, Dell Precision, Lenovo ThinkPad --- Запуск модульных тестов --- test_calculate_total_prices (__main__.TestRK1Functions.test_calculate_total_prices) Тест 2: Проверка расчета суммарной стоимости (Задание A2) ... ok test_filter_departments_with_computers (__main__.TestRK1Functions.test_filter_departments_with_computers) Тест 3: Проверка фильтрации отделов (Задание A3) ... ok test_sort_computers_by_class (__main__.TestRK1Functions.test_sort_computers_by_class) Тест 1: Проверка сортировки компьютеров по классам (Задание A1) ... ok ----- Ran 3 tests in 0.000s OK sofagoraceva@MacBook-Air-Sofa Goryacheva_2025_Labs % </pre>			